

# **University of Asia Pacific**

**Department of CSE**

**Assignment On**

## **The First-Come, First-Served (FCFS)**

### **CPU Scheduling Algorithm**

**Course Title:** Operating System Lab

**Course Code:** CSE 402

**Submitted To**

Atia Rahman Orthi  
Lecturer  
Department of Computer Science  
and Engineering  
University of Asia Pacific

**Submitted By**

Sultan Ruhul Kuddus Nayan  
ID: 23101159  
2022-2023 Batch  
Department of CSE  
University of Asia Pacific

**Date of Submission: 2 August 2026**

**1. Code Implementation :** The FCFS algorithm was implemented in Python. Processes are sorted by arrival time, then completion, turnaround, and waiting times are computed sequentially, followed by a Gantt chart and average-time summary. The complete source code is shown below.

```
Welcome | FCFS Lab.py X
C: > Users > HP > Desktop > FCFS Lab.py
1  processes = [
2      ["p0", 3, 1],
3      ["p1", 5, 3],
4      ["p2", 2, 2],
5      ["p3", 1, 2],
6      ["p4", 6, 3]
7  ]
8
9  processes.sort(key=lambda x: x[1])
10
11  time = 0
12  result = []
13
14  print("Gantt Chart:")
15
16  for p in processes:
17      pid, at, bt = p
18
19      if time < at:
20          print(f"| Idle ({time}-{at}) ", end="")
21          time = at
22
23      start = time
24      ct = start + bt
25      tat = ct - at
26      wt = tat - bt
27
28      result.append([pid, at, bt, ct, tat, wt])
29
30      print(f"| {pid} ({start}-{ct}) ", end="")
31      time = ct
32
33  print("|")
34  |
35  print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
36
37
38  print("|")
39  |
40  print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
41
42  total_tat = 0
43  total_wt = 0
44
45  for r in result:
46      print(f"{r[0]}\t{r[1]}\t{r[2]}\t{r[3]}\t{r[4]}\t{r[5]}")
47      total_tat += r[4]
48      total_wt += r[5]
49
50  n = len(result)
51
52  print("\nAverage Turnaround Time =", total_tat / n)
53  print("Average Waiting Time =", total_wt / n)
```

## 2. Sample Input and Output:

```
PS D:\Python Programming\Microsoft VS Code> & "C:\Program Files\Python31
Gantt Chart:
| Idle (0-1) | P3 (1-3) | P2 (3-5) | P0 (5-6) | P1 (6-9) | P4 (9-12) |

Process AT      BT      CT      TAT      WT
P3      1        2        3        2        0
P2      2        2        5        3        1
P0      3        1        6        3        2
P1      5        3        9        4        1
P4      6        3       12        6        3

Average Turnaround Time = 3.6
Average Waiting Time = 1.4
PS D:\Python Programming\Microsoft VS Code> |
```

## 3. Discussion :

The implemented Python program successfully simulates the **First-Come, First-Served (FCFS)** CPU scheduling algorithm. Initially, the processes are sorted according to their **Arrival Time (AT)** to ensure that the process arriving first is executed first. During execution, the program checks whether the CPU remains idle before the arrival of the next process and displays the idle period in the Gantt Chart when necessary.

For each process, the program calculates the **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** using standard scheduling formulas. All calculated values are stored in a result list and displayed in a tabular format. Finally, the program computes the **Average Turnaround Time** and **Average Waiting Time**, which are useful for evaluating the overall performance of the scheduling algorithm.

The generated Gantt Chart clearly illustrates the order and duration of process execution, making it easier to understand how FCFS scheduling works. Although FCFS is simple to implement and guarantees that every process is executed in the order of arrival, it may lead to higher waiting times when a long process is executed before shorter ones. This phenomenon is known as the **Convoy Effect**, which can reduce overall system efficiency

**4. Conclusion :** The FCFS scheduling algorithm was successfully implemented and verified in Python. It correctly computes completion, turnaround, and waiting times, and visualizes execution order via a Gantt chart. Results confirm FCFS's theoretical behavior: simple and fair by arrival order, but prone to the convoy effect and higher average waiting time when short processes arrive after long ones. For the tested dataset, **average turnaround time** was **3.60** units and **average waiting time** was **1.40** units, reinforcing the value of burst-time-aware algorithms such as SJF or Round Robin when minimizing wait time matters.

## **5. GitHub Repository**

Complete source code for this and other Operating System Lab experiments:

<https://github.com/sultan36912/Operating-System-Lab..git>